BÁO CÁO THỰC HÀNH

LAB-01 | Cấu hình Kernel và khởi động hệ thống

Từ bộ công cụ biên dịch đến chuỗi U-Boot → Linux → BusyBox

ARMv7-A • QEMU vexpress-a9 • Linux 5.15

DANH SÁCH NHÓM THỰC HÀNH

Họ và tên	MSSV
Lê Tiến Thành	SE201799
Lê Tấn Lộc	SE201412
Nguyễn Quang Huy Đạt	SE201415
Trần Đình Khôi Nguyên	SE200558
Trần Trương Tuấn Vũ	SE196467

Đơn vị thực hiện: Nhóm 5 sinh viên • Môn: Embedded Linux Systems

Tổng hợp dữ liệu thực hành: 09–10/2026

Tất cả hình minh chứng terminal sử dụng từ bài thực hành của nhóm; sơ đồ được thiết kế lại cho báo cáo này.

TỔNG QUAN & PHẠM VI

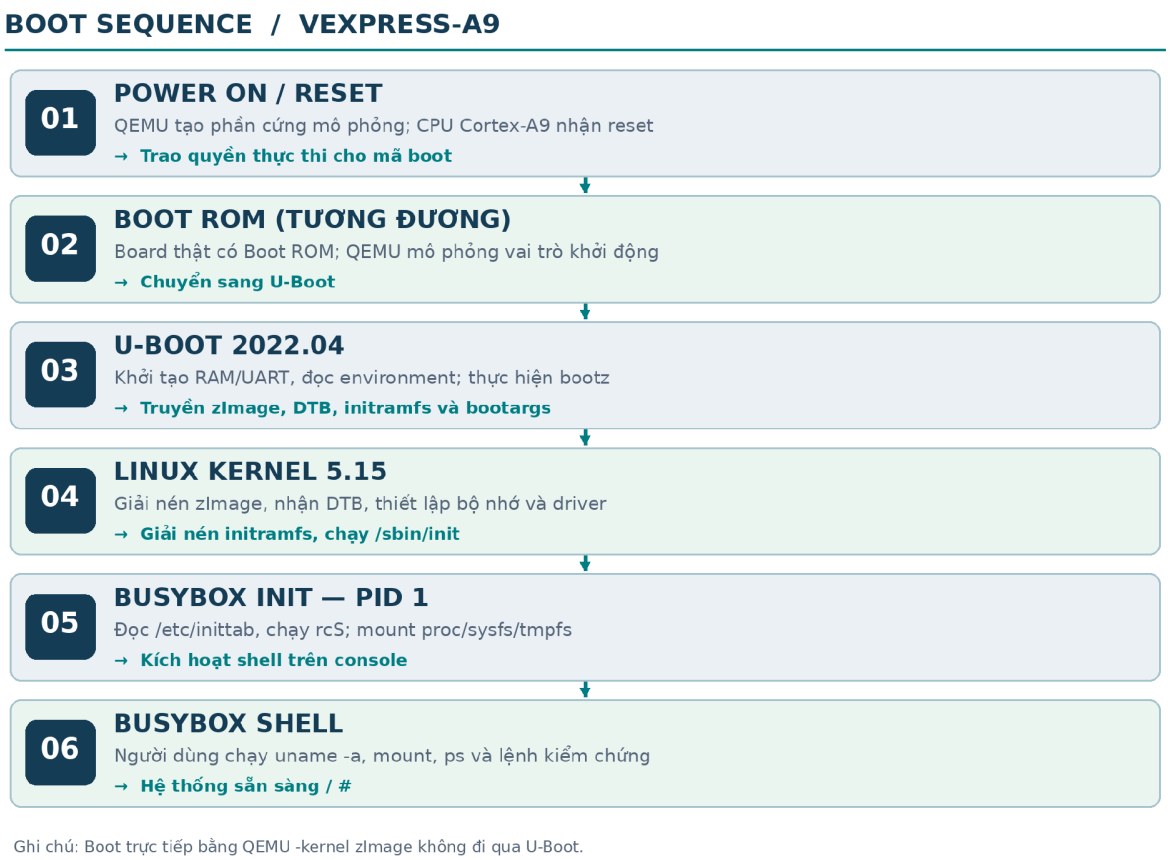
Bài thực hành được tổ chức thành ba nhánh: xây dựng kernel ARMv7 cho board Versatile Express, thao tác môi trường U-Boot và tạo root filesystem tối giản từ BusyBox. Thay vì chỉ kiểm tra đầu ra của lệnh build, nhóm đối chiếu file nhị phân với log khởi động thật, gồm cả lần clone repository LOS201 từ GitHub để thử khả năng tái lập.

Trụ cột	Thành phần / dữ liệu nhóm sử dụng
Kernel	Linux 5.15.0; zImage và vexpress-v2p-ca9.dtb; CONFIG_ARCH_VEXPRESS=y
Firmware khởi động	U-Boot 2022.04; giao diện serial và lệnh bootz
Userspace	BusyBox 1.35.0 (static ARM EABI5); initramfs.cpio.gz; /sbin/init
Phần cứng giả lập	QEMU vexpress-a9; Cortex-A9; RAM 512 MiB; UART PL011 ttyAMA0
Toolchain	arm-linux-gnueabihf-; xây dựng cho ARMv7-A trên Ubuntu host

Đọc kết quả đúng cách Lệnh QEMU -kernel zImage kiểm tra đường boot trực tiếp; muốn chứng minh có U-Boot trong chuỗi phải nạp U-Boot trước, sau đó dùng bootz để chạy kernel.

1. KIẾN TRÚC KHỞI ĐỘNG VÀ LUỒNG BÀN GIAO

Các tầng boot không phải những chương trình chạy đồng thời. Mỗi tầng thiết lập trạng thái tối thiểu, nhận hoặc tạo dữ liệu đầu vào rồi bàn giao cho tầng tiếp theo. Trên board vật lý có Boot ROM; trong QEMU bước khởi đầu được mô phỏng, vì thế cần phân biệt logic khởi động với phần cứng hiện hữu.



Hình 1. Sơ đồ tự thiết kế: sáu tầng khởi động, trách nhiệm và dữ liệu bàn giao.

U-Boot tạo môi trường cho Linux bằng cách cung cấp boot arguments cùng vị trí ba hiện vật: zImage, initramfs và DTB. Kernel dựa vào Device Tree để nhận diện phần cứng, giải nén initramfs rồi chạy /sbin/init. Sau khi /etc/inittab kích hoạt rcS, hệ thống mới mở /bin/sh để nhận lệnh.



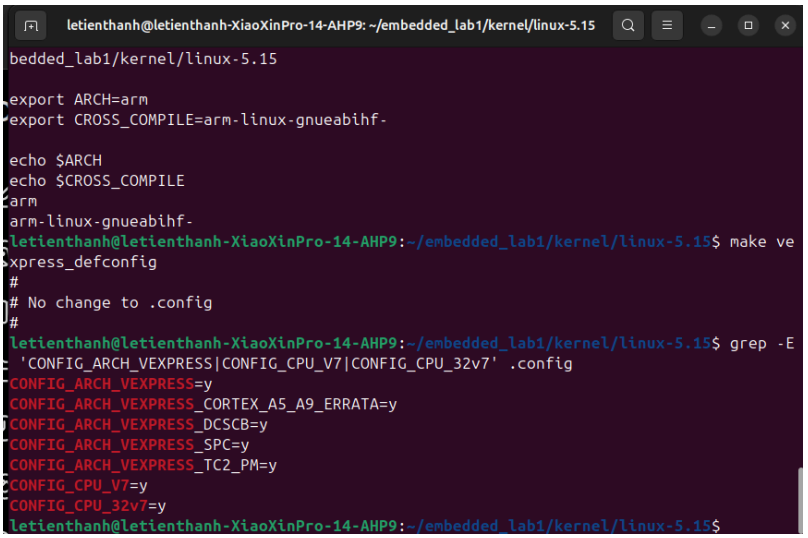
Hình 2. Địa chỉ nạp của phép thử U-Boot → Linux thành công ngày 21/10/2026.

2. PHẦN A — BIÊN DỊCH VÀ CẤU HÌNH KERNEL

2.1. Tại sao dùng vexpress_defconfig?

Nền tảng bài là vexpress-a9 (Cortex-A9, ARMv7) nên nhóm chọn vexpress_defconfig. Tùy chọn versatile_defconfig được đề cập trong checklist nhưng hướng đến board Versatile khác. Nhóm kiểm tra lại các cờ quan trọng thay vì chỉ dựa vào tên file cấu hình.

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
make vexpress_defconfig
grep -E "CONFIG_ARCH_VEXPRESS|CONFIG_CPU_V7|CONFIG_CPU_32v7" .config
```



Hình 3. Log của nhóm: đặt ARCH, toolchain và kiểm tra cấu hình ARMv7.

2.2. Những tùy chọn cần thiết

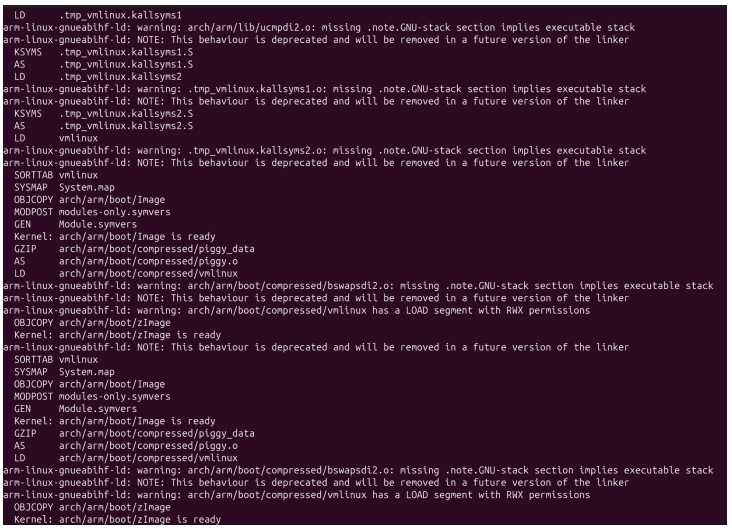
Cấu hình	Ý nghĩa khi boot
CONFIG_ARCH_VEXPRESS=y	Kích hoạt hỗ trợ platform vexpress.

CONFIG_BLK_DEV_INITRD=y	Cho phép dùng initramfs/initrd trong quá trình boot.
CONFIG_SERIAL_AMBA_PL011=y	Có driver UART PL011 phục vụ console ttyAMA0.
CONFIG_DEVTMPFS=y	Hỗ trợ filesystem thiết bị trên Linux.
CONFIG_SMP=y	Kernel có khả năng quản lý nhiều CPU; số CPU chạy thực tế còn phụ thuộc QEMU.

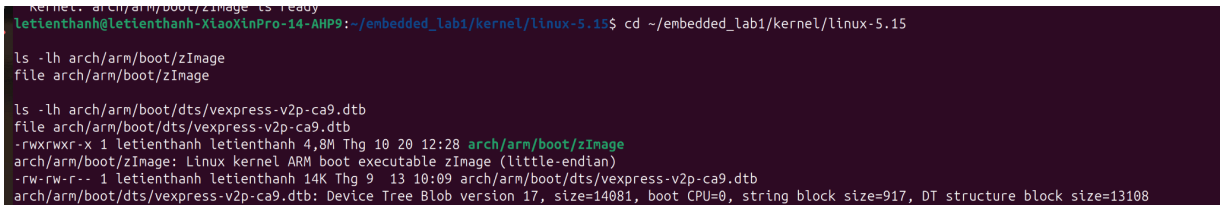
Menuconfig còn được dùng để quan sát cấu hình LPAE, Memory split 2G/2G và các tùy chọn CPU. LPAE không cần thiết cho cấu hình 512 MiB của thử nghiệm này, nhưng sự vắng mặt của LPAE không phải nguyên nhân duy nhất giải thích quyết định lựa chọn kernel.

2.3. Đầu ra bản build

```
make -j$(nproc) zImage dtbs modules
file arch/arm/boot/zImage
ls -lh arch/arm/boot/zImage
```



Hình 4. Phần cuối quá trình build Kernel từ log nhóm.



Hình 5. Nhận dạng và kích thước zImage/DTB.

Bản clone GitHub đã vượt qua phép kiểm tra định dạng: output/zImage là “Linux kernel ARM boot executable zImage (little-endian)”; file DTB, configs/kernel.config và configs/busybox.config tồn tại. Đây là bằng chứng về hiện vật được đóng gói, còn việc kernel chạy hay không được xác nhận ở phần tiếp theo.

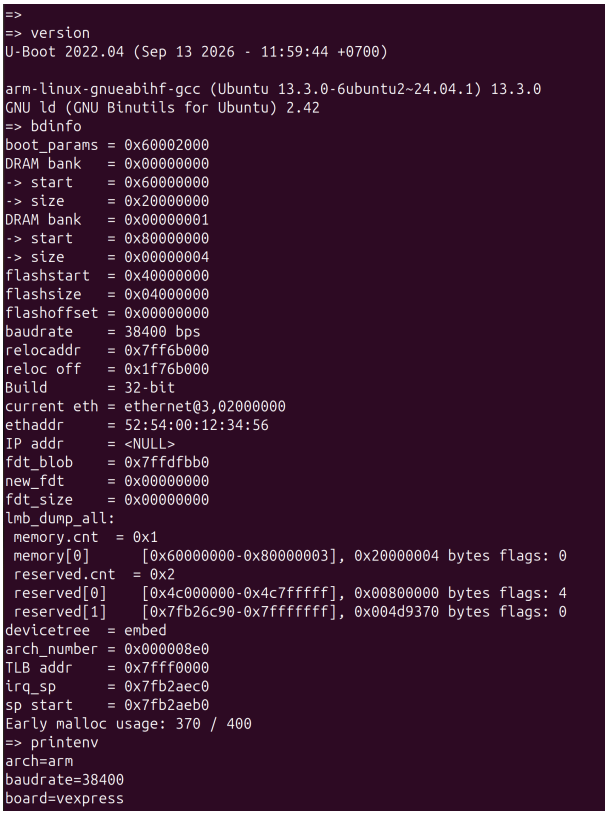
3. PHẦN B — THAO TÁC VỚI U-BOOT

3.1. Khởi động firmware và đọc môi trường

U-Boot 2022.04 được nạp trên máy QEMU vexpress-a9 và đưa người thực hành đến dấu nhắc “=>”. Firmware nhận diện DRAM 512 MiB, Flash 64 MiB và thiết bị Ethernet. Cảnh báo bad CRC cho thấy không có môi trường flash lưu hợp lệ; chương trình quay về default environment nhưng vẫn sử dụng được.

```
qemu-system-arm -M vexpress-a9 -m 512M -kernel output/u-boot -nographic
```

```
version
bdinfo
printenv bootcmd
printenv bootargs
help bootz
```



```
=>
=> version
U-Boot 2022.04 (Sep 13 2026 - 11:59:44 +0700)

arm-linux-gnueabihf-gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
GNU ld (GNU Binutils for Ubuntu) 2.42
=> bdinfo
boot_params = 0x60002000
DRAM bank   = 0x00000000
-> start    = 0x60000000
-> size     = 0x20000000
DRAM bank   = 0x00000001
-> start    = 0x80000000
-> size     = 0x00000004
flashstart  = 0x40000000
flashsize   = 0x04000000
flashoffset = 0x00000000
baudrate    = 38400 bps
relocaddr   = 0x7ff6b000
reloc off   = 0x1f76b000
Build       = 32-bit
current eth = ethernet@3,02000000
ethaddr     = 52:54:00:12:34:56
IP addr     = <NULL>
fdt_blob    = 0x7ffdfbb0
new_fdt     = 0x00000000
fdt_size    = 0x00000000
lmb_dump_all:
 memory.cnt  = 0x1
 memory[0]      [0x60000000-0x80000003], 0x20000004 bytes flags: 0
 reserved.cnt  = 0x2
 reserved[0]    [0x4c000000-0x4c7fffff], 0x00800000 bytes flags: 4
 reserved[1]    [0x7fb26c90-0x7fffffff], 0x004d9370 bytes flags: 0
devicetree  = embed
arch_number = 0x000008e0
TLB addr    = 0x7fff0000
irq_sp      = 0x7fb2aec0
sp start    = 0x7fb2aeb0
Early malloc usage: 370 / 400
=> printenv
arch=arm
baudrate=38400
board=vexpress
```

Hình 6. Log U-Boot của nhóm: version, bdinfo, printenv.

3.2. Vai trò của bootargs và bootcmd

bootargs là chuỗi kernel command line (ví dụ console=ttyAMA0,115200 và rdinit=/sbin/init); bootcmd là chuỗi lệnh mà U-Boot sẽ thực thi để boot. Cú pháp bootz yêu cầu địa chỉ zImage, địa chỉ raw initrd có kèm kích thước, và địa chỉ FDT. Lưu giá trị môi trường bằng saveenv không cần thiết cho việc kiểm thử tạm thời trong QEMU.

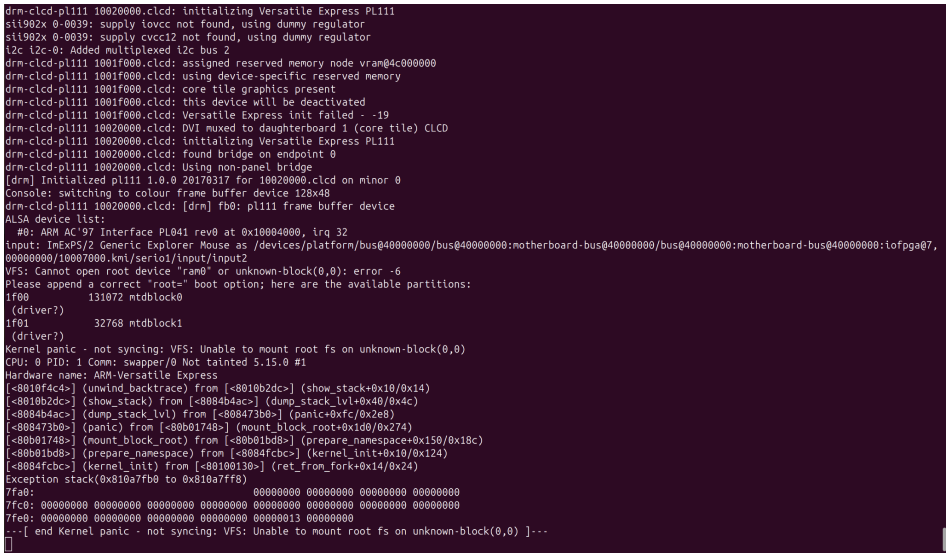
```
setenv bootargs 'console=ttyAMA0,115200 rdinit=/sbin/init'
setenv bootcmd 'bootz 0x62000000 0x64000000:0x1a0000 0x66000000'
printenv bootargs
printenv bootcmd
```

Trong lần thực nghiệm thành công, zImage được nạp tại 0x62000000, archive initramfs tại 0x64000000 và DTB tại 0x66000000. Trước đó một lần thử với DTB tại 0x60000000 dẫn đến FDT_ERR_BADSTRUCTURE; nhóm chuyển vị trí DTB và xác nhận lại bằng fdt header (magic 0xd00dfeed). Không khẳng định chắc chắn nguyên nhân xung đột bộ nhớ khi chưa kiểm tra sâu.

Thông số cần tái kiểm tra Giá trị 0x1a0000 là độ dài initrd sử dụng trong log boot đạt. Nhật ký đính kèm chưa có kết quả stat chứng minh kích thước thật của file cpio.gz; khi tái chạy cần dùng đúng số byte tính từ stat, không mặc định lấy giá trị này.

3.3. Thử nghiệm không có rootfs

Bài kiểm tra âm tính loại bỏ root filesystem để quan sát giới hạn trách nhiệm giữa Kernel và userspace. Kernel vẫn có thể đi tới giai đoạn nhận diện hệ thống nhưng không mount được root filesystem phù hợp; nhật ký cũ ghi nhận Kernel panic trước khi vào BusyBox shell.

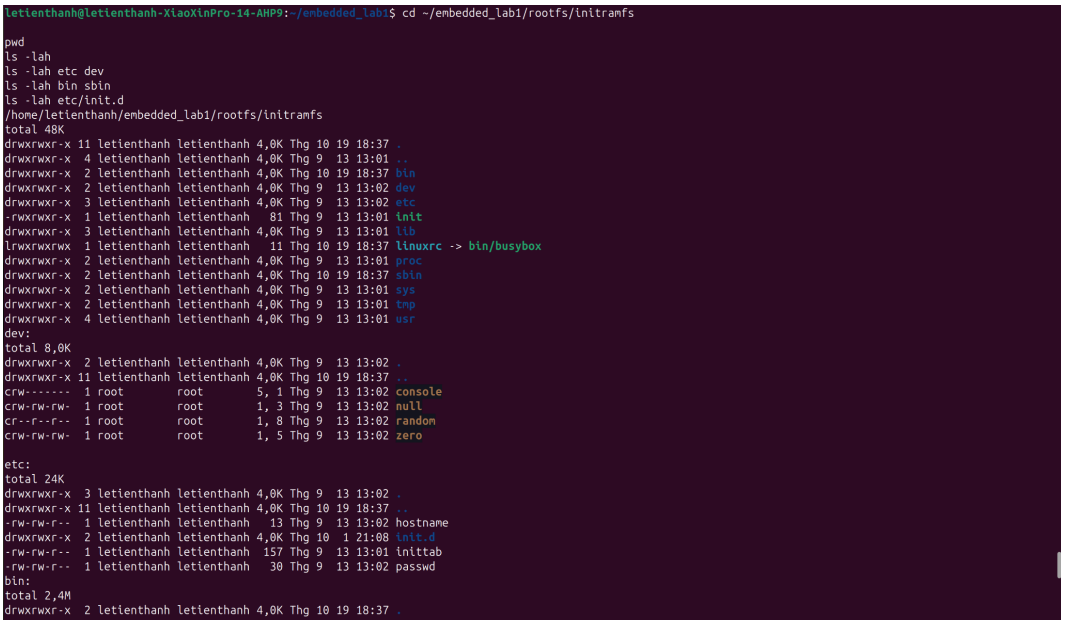


Hình 7. Ảnh thực nghiệm của nhóm: kernel panic khi thiếu rootfs.

4. PHẦN C — BUSYBOX, INITRAMFS VÀ KẾT QUẢ BOOT

4.1. Thiết kế root filesystem tối giản

BusyBox được biên dịch kiểu static cho ARM EABI5, vì vậy /sbin/init và các applet không phải dựa vào dynamic loader cùng thư viện dùng chung trong image gốc. Initramfs gồm các nhánh bin, sbin, etc, dev, proc, sys, tmp; nhóm tạo device node console, null, zero và random để phục vụ môi trường cơ bản.



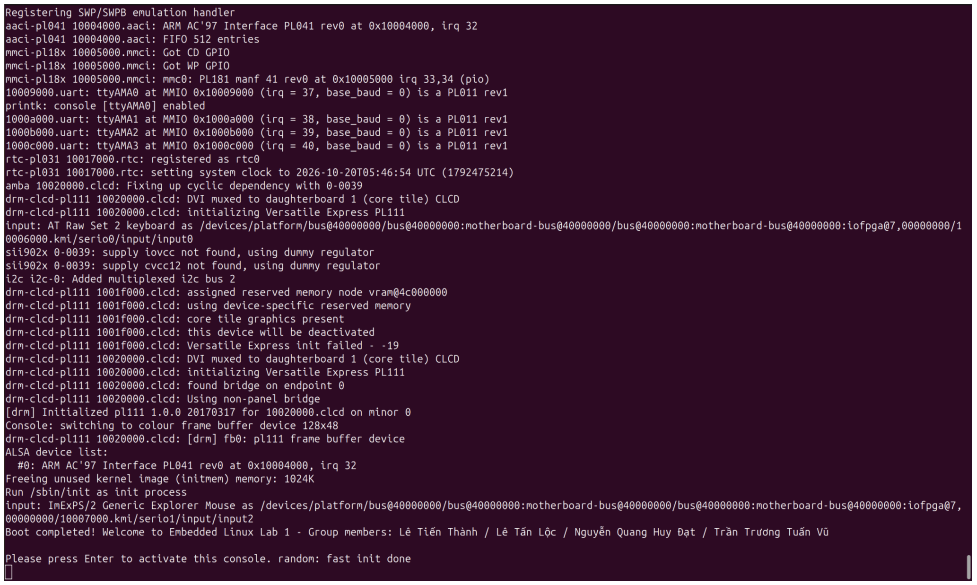
Hình 8. Cấu trúc và node thiết bị trong rootfs của nhóm.

```
cd rootfs/initramfs
find . | cpio -H newc -o | gzip > ../../output/initramfs.cpio.gz
```

Từ giao diện boot, kernel giải nén archive vào RAM rồi chạy /sbin/init. Theo file cấu hình init, BusyBox gọi rcS để mount /proc, /sys, tmpfs ở /tmp và khởi tạo console.

4.2. Khởi động trực tiếp Kernel trên QEMU

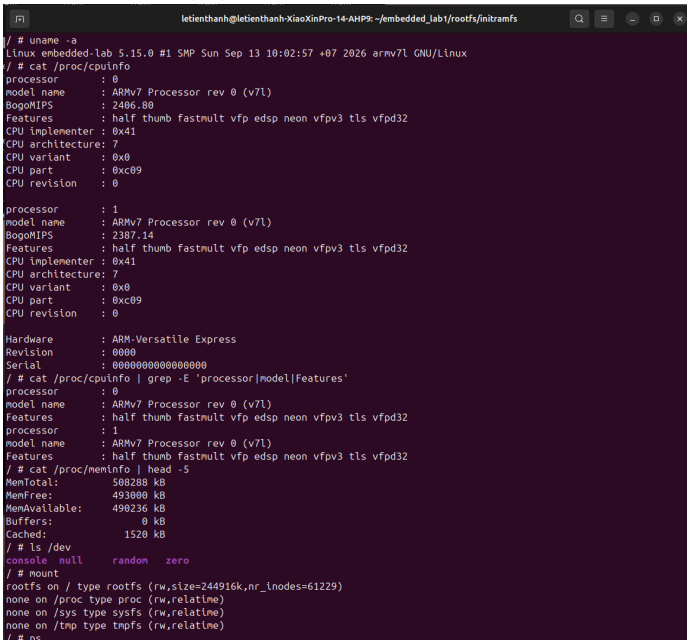
```
qemu-system-arm -M vexpress-a9 -m 512M \
  -kernel output/zImage -dtb output/vexpress-v2p-ca9.dtb \
  -initrd output/initramfs.cpio.gz \
  -append "console=ttyAMA0 rdinit=/sbin/init" -nographic
```



Hình 9. Kernel/BusyBox khởi động tới console shell trên vexpress-a9.

4.3. Đối chiếu trạng thái sau boot

Lệnh	Quan sát của nhóm	Ý nghĩa
uname -a	Linux embedded-lab 5.15.0; armv7l	Đúng kernel và kiến trúc mục tiêu
mount	rootfs /; proc /proc; sysfs /sys; tmpfs /tmp	Init hoàn tất mount cơ bản
ps	init có PID 1; shell -/bin/sh	Userspace đã khởi chạy
cat /proc/cpuinfo	ARMv7; CPU part 0xc09	Nhận diện Cortex-A9
cat /proc/cmdline	console=ttyAMA0 rdinit=/sbin/init	Kernel nhận tham số dòng lệnh



Hình 10. Terminal nhóm: uname, CPU info và filesystem sau boot.

5. TÁI KIỂM THỬ TỪ REPOSITORY GITHUB

Nhóm clone lại LOS201 từ origin thay vì chạy trên cây mã nguồn build. Bản clone có branch main và clean working tree; kiểm tra Git index cho thấy Lab 1 không còn được lưu bằng gitlink/submodule. Những hiện vật bắt buộc output/zImage, DTB, initramfs và hai cấu hình đều tồn tại, đủ để boot mà không cần chứa toàn bộ thư mục source kernel 2,2 GiB.

```
git clone https://github.com/lethanh-new/LOS201.git LOS201_LAB1_TEST
cd LOS201_LAB1_TEST/embedded_lab1
file output/zImage output/u-boot rootfs/initramfs/bin/busybox
# file xác nhận zImage ARM, U-Boot ELF32 ARM, BusyBox ARM static
```

Đường boot qua U-Boot đã được kiểm chứng: U-Boot nhận zImage ở 0x62000000 và DTB ở 0x66000000, báo “Loading Ramdisk … OK”, “Loading Device Tree … OK”, “Starting kernel …”. Trong Linux xuất hiện “Unpacking initramfs…”, “Run /sbin/init as init process” và banner Welcome của bài lab; sau đó chạy được uname -a, ps, cat /proc/cmdline.

```
Kernel image @ 0x62000000
## Flattened Device Tree blob at 66000000
Loading Ramdisk ... OK
Loading Device Tree ... OK
Starting kernel ...
Unpacking initramfs...
Run /sbin/init as init process
Boot completed! Welcome to Embedded Linux Lab 1 ...
/ #
```

Kết quả kiểm thử Chức năng boot chính PASS ở cả hai đường: QEMU nạp trực tiếp Kernel và U-Boot gọi bootz. Tuy nhiên báo cáo cần được xuất PDF và bổ sung vào bao_cao/ trên GitHub để hoàn thiện yêu cầu nộp bài.

6. GIẢI THÍCH 5 VẤN ĐỀ KỸ THUẬT

6.1. ABI gnueabi và gnueabihf

Khác biệt cốt lõi là quy ước truyền tham số số thực qua thanh ghi. gnueabi dùng base procedure-call variant không truyền float qua VFP registers, trong khi gnueabihf dùng hard-float procedure-call standard cho VFP. Đây là tính chất ABI chứ không nên đồng nhất tuyệt đối với việc CPU không chạy lệnh FPU. Kernel tự tránh sử dụng floating-point trong kernel mode; tính tương thích userspace đặc biệt quan trọng khi sử dụng thư viện động.

6.2. Tại sao RAM bắt đầu ở 0x60000000?

Bố trí vật lý Versatile Express khác Versatile PB: một số địa chỉ thấp dành cho thiết bị/mạch chủ, trong khi vùng RAM được cấp cho vexpress-a9 bắt đầu tại 0x60000000. Vì vậy địa chỉ load zImage/DTB/initramfs phải được chọn trong RAM, không chồng lên vùng dành riêng hoặc chồng nội dung của nhau.

6.3. Phân tích mã MIDR 0x410fc090

Implementer 0x41 chỉ ARM; variant 0x0; architecture field 0xF sử dụng cơ chế nhận dạng mở rộng; part number 0xC09 là Cortex-A9; revision 0. CPU part trong /proc/cpuinfo (0xc09) phù hợp với kết quả này.

6.4. Lý do BusyBox cần static linking trong image nhỏ

Static binary mang theo các phần mã thư viện cần sử dụng vào file chạy, giúp initramfs tránh phải đóng gói loader động và bộ thư viện phụ thuộc. Đổi lại kích thước executable có thể lớn hơn. Nếu sử dụng BusyBox dynamic, phải kiểm tra ELF interpreter và tất cả mục NEEDED bằng readelf rồi chép đủ vào rootfs.

6.5. SMP và số CPU thật được kích hoạt

Bật CONFIG_SMP chỉ cho biết Kernel hỗ trợ multiprocessor. Số CPU online bị chi phối bởi mô hình QEMU, tham số -smp và luồng khởi tạo CPU phụ. Trong lần boot qua U-Boot không đặt -smp, log báo “SMP: Total of 1 processors activated”; điều này không mâu thuẫn với kernel đã biên dịch hỗ trợ SMP. Nhật ký cũ của nhóm có thử -smp 1 và -smp 2. Giá trị possible=0-3 phản ánh tập CPU khả dĩ, không tương đương tập CPU đang online.

7. KHÓ KHĂN, GIỚI HẠN VÀ KẾT LUẬN

Vướng mắc thực tế	Cách nhóm xử lý / điều cần làm rõ
menuconfig hiển thị x86	Export ARCH=arm và CROSS_COMPILE, sau đó xác minh CONFIG_CPU_V7 trước khi build.
Thiếu các flag cấu hình ARMv7 trong lần xem ban đầu	Chạy lại defconfig phù hợp; grep .config để kiểm tra trước khi biên dịch.
bootz báo FDT_ERR_BADSTRUCTURE	Nạp lại DTB ở vùng RAM khác, dùng fdt header xác nhận tính hợp lệ; không khẳng định nguyên nhân nếu thiếu phép đo.
Kiểm thử có thể nhầm SMP hỗ trợ với CPU đang chạy	Đối chiếu -smp, /proc/cpuinfo, /sys/devices/system/cpu/online và boot log.

Kết luận: Dữ liệu từ chính nhóm xác nhận kernel Linux 5.15 ARMv7, firmware U-Boot 2022.04 và rootfs BusyBox có thể boot thành công trên QEMU vexpress-a9. Cả đường boot trực tiếp lẫn đường boot đầy đủ qua U-Boot đều được kiểm thử trên bản clone từ GitHub. Cần đính kèm báo cáo PDF và kiểm tra lại độ dài initramfs bằng stat trước khi công bố lệnh bootz tái lập tuyệt đối.

Tài liệu đối chiếu: Đề LAB-01; Linux 5.15 Documentation; U-Boot 2022.04; BusyBox 1.35.0; QEMU ARM; Lab01_Report.pdf (tham khảo bố cục/chủ đề, không dùng kết quả nhóm khác).